

# Session 3: Git and $\text{\LaTeX}$ in VS Code

## Version Control and the Integrated Research Environment

Brady Allardice

UPF & IBEI

**Session 1:** What Claude Code can do. How it works. CLAUDE.md and plan mode.

**Session 2:** The core workflow. Data cleaning, estimation, robustness,  $\text{\LaTeX}$  tables.

**Session 3:** Building infrastructure around the workflow.

Today you will:

- 1 Learn the minimum viable git workflow for agentic coding
- 2 Bring your paper into the same project directory as your code
- 3 Use Claude Code to write, compile, and version-control a  $\text{\LaTeX}$  document

Session 2 taught you to verify Claude Code's work. Today you get the tools to manage it.

Git is research insurance,  
not software engineering ceremony.

You need exactly four commands.

# What Are Git and GitHub?

**Git** is a tool that tracks every change to every file in a project. It runs on your computer. It is free and open source.

Think of it as “track changes” for your entire project directory — not just one document, but every file: code, data descriptions,  $\text{\LaTeX}$  files, everything.

**GitHub** is a website that hosts git projects online.

- It is where open-source code, replication packages, and increasingly papers live
- Git works entirely on your machine. GitHub is optional — it adds backup, sharing, and collaboration
- Today we only use git locally. GitHub comes in Session 4.

## The Key Idea

Git lets you save snapshots of your project at any point. You can always see what changed between snapshots, and go back to any previous one.

# Why Researchers Should Care About Git

## Without git, you have probably done this:

- `analysis_v2.py`, `analysis_v2_final.py`, `analysis_v2_final_REAL.py`
- “Which version of the cleaning script produced these results?”
- “I changed something last week and now the code is broken”

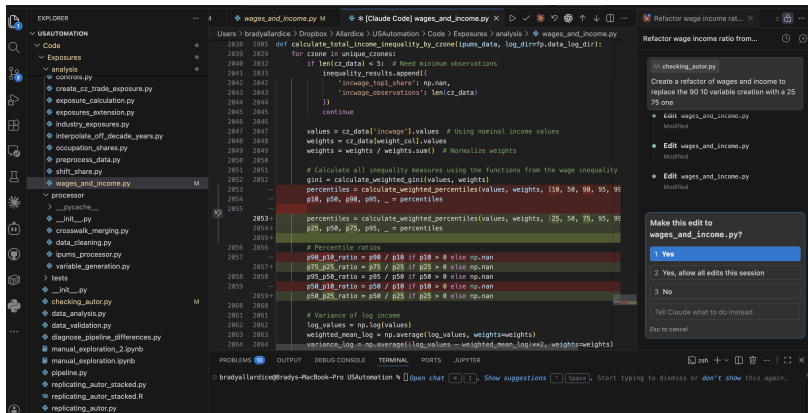
## With git:

- One file, full history. Every version is recoverable.
- You can see exactly what changed, when, and why.
- You can undo any change — even from weeks ago.

Git is useful for any research project. But when you are working with an agent that modifies your files directly, it goes from useful to **essential**.

# You Have Already Seen This

## In VS Code, when Claude Code proposes a change, you see a diff view:



Red = removed, green = added. You have been reading diffs since Session 1.

**Git does the same thing** — but for your entire project, across every change, and you can undo any of them after the fact.

# Why Git Matters for Agentic Coding

**The problem:** Claude Code modifies your files directly.

- It might change a file you did not ask it to change
- It might overwrite working code with something broken
- It might make 10 changes when you only wanted 3

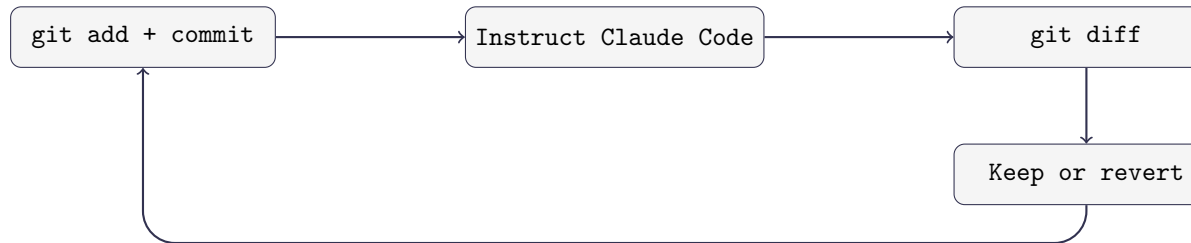
**Without git:** You try to manually undo changes. You are not sure what the file looked like before. You lose work.

**With git:** You see exactly what changed, line by line. You revert anything you do not want. You never lose work.

## The Rule

Never let Claude Code touch your project without a recent commit to fall back on.

# The Minimum Viable Git Workflow



---

## Command

## What it does

---

```
git add -A && git commit -m "msg"
```

Snapshot your current state

```
git diff
```

See what changed since the last commit

```
git checkout -- file.py
```

Undo changes to a specific file

```
git log --oneline
```

See your commit history

---

## Step 1: Commit Before You Instruct

**Always save your working state before Claude Code touches anything.**

```
git add -A  
git commit -m "Working merge script before adding controls"
```

**What this gives you:**

- A snapshot you can always return to
- A clear “before” to compare against
- Permission to let Claude Code experiment — you can always undo

### The Mindset Shift

Commits are not milestones. They are save points. Commit early, commit often, commit before every interaction with Claude Code.

## Step 2: Review the Diff

After Claude Code makes changes, see exactly what it did:

```
git diff
```

### What to look for:

- Files you did not ask it to change
- Lines removed that should not have been
- “Helpful” additions you did not request

### You can also ask Claude Code to explain the diff:

```
Run git diff and explain each change you made.
```

**Read the diff, not just the output.** The diff shows you what actually changed.

## Step 3: Keep or Revert

### If everything looks good:

```
git add -A  
git commit -m "Add demographic controls"
```

### If something is wrong — revert a specific file:

```
git checkout -- scripts/analysis.py
```

### If everything is wrong — revert all changes:

```
git checkout -- .
```

## The 30-Second Safety Net

This entire cycle takes 30 seconds of overhead. That is the cost of never losing work.

## Exercise Part 1: Initialize and Commit (10 min)

- ❶ **Initialize git** in your project directory (if you have not already):
  - `git init`
  - `git add -A && git commit -m "Initial commit"`
- ❷ **Commit** your current work from Sessions 1–2.
  - `git add -A && git commit -m "Session 2 work: merge, estimation, robustness"`
- ❸ **Run** `git log --oneline` to see your history.

You now have a snapshot of everything that works. Whatever happens next, you can always come back here.

## Exercise Part 2: The Cycle (25 min)

Do this **at least three times**:

- ❶ **Commit** your current state
- ❷ **Instruct Claude Code** to make a change:
  - Round 1: Add a new control variable to your specification
  - Round 2: Refactor the analysis script into functions
  - Round 3: Change the sample restriction
- ❸ **Run** `git diff` and read what changed
- ❹ **Decide**: keep (commit) or revert (`git checkout -- .`)

**Try to revert at least once.** The point is to see that you can always undo.

Commit first, instruct second, diff third, decide fourth.

If you take one thing from this module, it is this sequence.

## Discussion:

- Did Claude Code change any files you did not expect?
- Was the diff easy to read? What was confusing?
- Did anyone revert a change? What went wrong?

## The Habit

**Commit** → **Instruct** → **Diff** → **Decide**.

This is not optional when working with an agent that modifies your files. It is the minimum responsible workflow.

In Session 4, we will automate the commit step with a hook — so you never forget.

What if your paper lived in the same place  
as your code and data?

This is the conceptual pivot of the course.

# The Problem with Overleaf

**Most researchers keep their paper in a separate tool.**

- Code lives in a project directory (or on your Desktop)
- Data lives next to the code (maybe)
- The paper lives in Overleaf (or Word, or Google Docs)

**What this means for Claude Code:**

- It cannot read your paper
- It cannot check the methods section against your code
- It cannot update a table when the analysis changes
- It cannot commit changes to your paper alongside code changes

## The Limitation

If Claude Code cannot see it, Claude Code cannot operate on it.

Your paper is the biggest piece of your project that is currently invisible.

# The Solution: $\text{\LaTeX}$ in VS Code

## Bring your paper into the project directory.

- Your `.tex` file lives alongside `scripts/` and `data/`
- Claude Code can read it, write it, and edit it — just like any other file
- Changes to the paper are tracked by git, just like code changes
- Coauthors review changes through version control, not “track changes”

## What becomes possible:

- Claude Code writes a methods section and you verify it against the code
- The analysis changes  $\rightarrow$  the table updates  $\rightarrow$  the paper updates
- Every version of the paper is recoverable through git

## The Point

This is not about  $\text{\LaTeX}$  vs. Word. It is about bringing your paper into the agentic workflow.

## What you need (should be installed from pre-work):

- ① A T<sub>E</sub>X distribution: TeX Live (Linux/Windows) or MacTeX (Mac)
- ② The **LaTeX Workshop** extension in VS Code

## What you get:

- Syntax highlighting for `.tex` files
- One-click compilation to PDF
- PDF preview in a split pane — side by side with your code
- Error messages in the VS Code terminal

## The workflow:

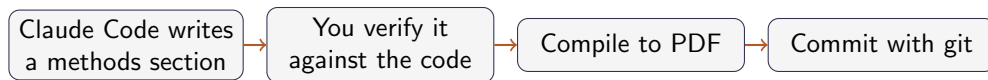
Edit `.tex` → Save → Auto-compile → PDF updates in the preview pane.

You never leave VS Code. Your paper is right next to your code.

```
\documentclass{article}
\usepackage{booktabs}
\title{My Research Paper}
\author{Your Name}
\begin{document}
\maketitle
\section{Data Description}
We use a survey of 2,044 Polish adults...
\section{Results}
\input{output/robustness_table.tex}
\end{document}
```

**Key idea:** `\input{output/robustness_table.tex}` pulls in the table from Session 2. When the table updates, the paper updates.

# Demo: The Full Loop



Everything happens in VS Code.

Everything is version-controlled.

Everything is visible to Claude Code.

## Exercise Part 3: Build a $\text{\LaTeX}$ Document (30 min)

Using the provided template:

- 1 **Commit** your current state (the habit starts now)
- 2 **Ask Claude Code** to create a `.tex` file with:
  - A data description section based on your Session 2 pipeline
  - The robustness table via `\input{output/robustness_table.tex}`
- 3 **Compile** to PDF and verify it renders correctly
- 4 **Diff and commit** the new file
- 5 **Ask Claude Code** to write a results paragraph interpreting the table. Review what it writes — would you sign off on this for a paper?
- 6 **Ask Claude Code** to check the data description against the actual cleaning code. Does the text match what the code does?

Notice: you are now using git and  $\text{\LaTeX}$  together. Every change to the paper is tracked.

# What You Built Today

- 1 **Git:** You have a safety net. Commit → instruct → diff → decide. You practiced reverting changes and saw that you can always go back.
- 2 **L<sup>A</sup>T<sub>E</sub>X in VS Code:** Your paper is now part of your project. Claude Code can see it, edit it, and check it against your code.
- 3 **The two together:** Every change to your paper is version-controlled, just like every change to your code. You used the git cycle on your `.tex` file.

## The Bigger Picture

Your project directory now contains: data, code, output, and a paper — all under version control, all visible to Claude Code. This is the foundation for everything in Session 4.

## Discussion questions:

- 1 Did Claude Code change any files you did not expect? How did you handle it?
- 2 Did anyone revert a change? What went wrong?
- 3 When Claude Code wrote parts of the  $\text{\LaTeX}$  document, what did you have to correct?
- 4 How does this compare to your current workflow — Overleaf, Word, separate directories?

## Key Takeaway

The skill is not just using Claude Code. It is building an environment where Claude Code can do its best work — and where you can verify and undo everything it does.

### Session 4: Power Features — Hooks, MCPs, and Skills

- **Hooks:** Automate the commit step — every change Claude Code makes is tracked automatically
- **MCPs:** Connect Zotero — search your library and pull citations from inside Claude Code
- **Skills:** Reusable workflows — spec validation, robustness checks, and more

#### Before next session:

- Practice the commit-instruct-diff-decide cycle on your own project
- Try asking Claude Code to review a piece of your own code
- Make sure your Zotero account is set up (instructions in the setup guide)